11.1.4 Comparing enforcement patterns

ACME ended up using all three enforcement patterns in different situations. The API gateway managed edge traffic, the service mesh secured most east–west calls, and per-service proxies protected APIs that had not yet migrated into the mesh. Each pattern offered distinct advantages and trade-offs, summarized in table 11.1.

Table 11.1 Comparison of API authorization enforcement patterns

Pattern	Where enforced	Strengths
API gateway	At the edge, all incoming requests funnel through the gateway	Centralized enforcement, consistent logs, easy to roll out global rules
Service mesh with sidecars	Between services inside the cluster; each call passes through local sidecars	Strong workload identity via mTLS, zero-trust baseline, no code changes required
Per-service proxies	In front of a single API; proxy acts as a local PEP	Works without mesh, finer blast radius, consistent policy for legacy services

Together, these three patterns gave ACME a consistent, Cedar-based authorization model across their growing API ecosystem. Regardless of whether a request passed through the gateway, a mesh sidecar, or a per-service proxy, the decision was always externalized to Cedar and evaluated in the same PARC format.

11.1.5 Other enforcement patterns

The gateway, mesh, and per-service proxy patterns addressed most of ACME's needs. However, in certain cases, other enforcement models can also be useful. These are less common in ACME's story but still worth understanding.

IN-PROCESS ENFORCEMENT

Instead of routing requests through a proxy, a service can integrate the Cedar library or run a local Cedar agent. This decreases latency and eliminates an extra network hop. It's especially beneficial for performance-critical services